

- Bratley, P., Fox, B.L., and Schrage, E.L. 1983, *A Guide to Simulation*, 2nd ed. (New York: Springer).[7].
- Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), pp. 125ff.[8]

7.4 Multivariate Normal Deviates

A multivariate random deviate of dimension M is a point in M -dimensional space. Its coordinates are a vector, each of whose M components are random — but not, in general, independently so, or identically distributed. The special case of *multivariate normal deviates* is defined by the multidimensional Gaussian density function

$$N(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{M/2} \det(\boldsymbol{\Sigma})^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}) \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu})\right] \quad (7.4.1)$$

where the parameter $\boldsymbol{\mu}$ is a vector that is the mean of the distribution, and the parameter $\boldsymbol{\Sigma}$ is a symmetrical, positive-definite matrix that is the distribution's covariance.

There is a quite general way to construct a vector deviate $\mathbf{x}$ with a specified covariance $\boldsymbol{\Sigma}$ and mean $\boldsymbol{\mu}$, starting with a vector $\mathbf{y}$ of independent random deviates of zero mean and unit variance: First, use Cholesky decomposition (§2.9) to factor $\boldsymbol{\Sigma}$ into a left triangular matrix $\mathbf{L}$ times its transpose,

$$\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^T \quad (7.4.2)$$

This is always possible because $\boldsymbol{\Sigma}$ is positive-definite, and you need do it only once for each distinct $\boldsymbol{\Sigma}$ of interest. Next, whenever you want a new deviate $\mathbf{x}$, fill $\mathbf{y}$ with independent deviates of unit variance and then construct

$$\mathbf{x} = \mathbf{L}\mathbf{y} + \boldsymbol{\mu} \quad (7.4.3)$$

The proof is straightforward, with angle brackets denoting expectation values: Since the components y_i are independent with unit variance, we have

$$\langle \mathbf{y} \otimes \mathbf{y} \rangle = \mathbf{1} \quad (7.4.4)$$

where $\mathbf{1}$ is the identity matrix. Then,

$$\begin{aligned} \langle (\mathbf{x} - \boldsymbol{\mu}) \otimes (\mathbf{x} - \boldsymbol{\mu}) \rangle &= \langle (\mathbf{L}\mathbf{y}) \otimes (\mathbf{L}\mathbf{y}) \rangle \\ &= \left\langle \mathbf{L}(\mathbf{y} \otimes \mathbf{y})\mathbf{L}^T \right\rangle = \mathbf{L} \langle \mathbf{y} \otimes \mathbf{y} \rangle \mathbf{L}^T \\ &= \mathbf{L}\mathbf{L}^T = \boldsymbol{\Sigma} \end{aligned} \quad (7.4.5)$$

As general as this procedure is, it is, however, rarely useful for anything except multivariate *normal* deviates. The reason is that while the components of $\mathbf{x}$ indeed have the right mean and covariance structure, their detailed distribution is not anything “nice.” The x_i ’s are linear combinations of the y_i ’s, and, in general, a linear combination of random variables is distributed as a complicated convolution of their individual distributions.

For Gaussians, however, we do have “nice.” All linear combinations of normal deviates are themselves normally distributed, and completely defined by their mean and covariance structure. Thus, if we always fill the components of $\mathbf{y}$ with normal deviates,

$$y_i \sim N(0, 1) \quad (7.4.6)$$

then the deviate (7.4.3) will be distributed according to equation (7.4.1).

Implementation is straightforward, since the Cholesky structure both accomplishes the decomposition and provides a method for doing the matrix multiplication efficiently, taking advantage of $\mathbf{L}$ ’s triangular structure. The generation of normal deviates is inline for efficiency, identical to `Normaldev` in §7.3.

```
struct Multinormaldev : Ran { multinormaldev.h
  Structure for multivariate normal deviates.
  Int mm;
  VecDoub mean;
  MatDoub var;
  Cholesky chol;
  VecDoub spt, pt;

  Multinormaldev(Ullong j, VecDoub &mmean, MatDoub &vvar) :
  Ran(j), mm(mmean.size()), mean(mmean), var(vvar), chol(var),
  spt(mm), pt(mm) {
    Constructor. Arguments are the random generator seed, the (vector) mean, and the (ma-
    trix) covariance. Cholesky decomposition of the covariance is done here.
    if (var.ncols() != mm || var.nrows() != mm) throw("bad sizes");
  }

  VecDoub &dev() {
    Return a multivariate normal deviate.
    Int i;
    Doub u,v,x,y,q;
    for (i=0;i<mm;i++) { Fill a vector of independent normal deviates.
      do {
        u = doub();
        v = 1.7156*(doub()-0.5);
        x = u - 0.449871;
        y = abs(v) + 0.386595;
        q = SQR(x) + y*(0.19600*y-0.25472*x);
      } while (q > 0.27597
        && (q > 0.27846 || SQR(v) > -4.*log(u)*SQR(u)));
      spt[i] = v/u;
    }
    chol.elmult(spt,pt); Apply equation (7.4.3).
    for (i=0;i<mm;i++) {pt[i] += mean[i];}
    return pt;
  }
};
```

7.4.1 Decorrelating Multiple Random Variables

Although not directly related to the generation of random deviates, this is a convenient place to point out how Cholesky decomposition can be used in the reverse manner, namely to find linear combinations of correlated random variables that have no correlation. In this application we are given a vector $\mathbf{x}$ whose components have a known covariance Σ and mean μ . Decomposing Σ as in equation (7.4.2), we assert